一、图书管理系统

二、教学目标

1、掌握F和Q查询

2、掌握图书管理系统的项目创建

3、掌握多对多的查询和渲染

4、完成图书管理的列表CRUD

三、F和Q查询

models.py 中添加一下代码

```
#图书表
class Book(models.Model):
    .....

     #如果你想在已有的表 已有的数据 中添加字段
     #必须满足2个条件 要么添加default 要么允许为空 Null=True 不然会报错
     #卖出
     maichu = models.IntegerField(default=100)
     #库存
     kucun = models.IntegerField(default=100)

     .....
```

3.0.1 F对象

`F`对象用于在查询中引用其他字段。它可以用于查询条件、更新操作、聚合操作等。

`F`对象位于`Django`中存储在`django.db.models`模块中：

```
from django.db.models import F
```

`F`对象的语法结构是：

```
F('<field_name>')
```

3.0.1 Q对象

`Q`对象用于构建复杂查询条件的工具，可以将多个查询条件组合在一起，使用逻辑运算符进行连接，从而实现更加复杂的查询。`Q`对象位于`Django`中存储在`django.db.models`模块中：

```
from django.db.models import Q
```

`Q`对象的语法结构是：

```
Q(<query_expression>)
```

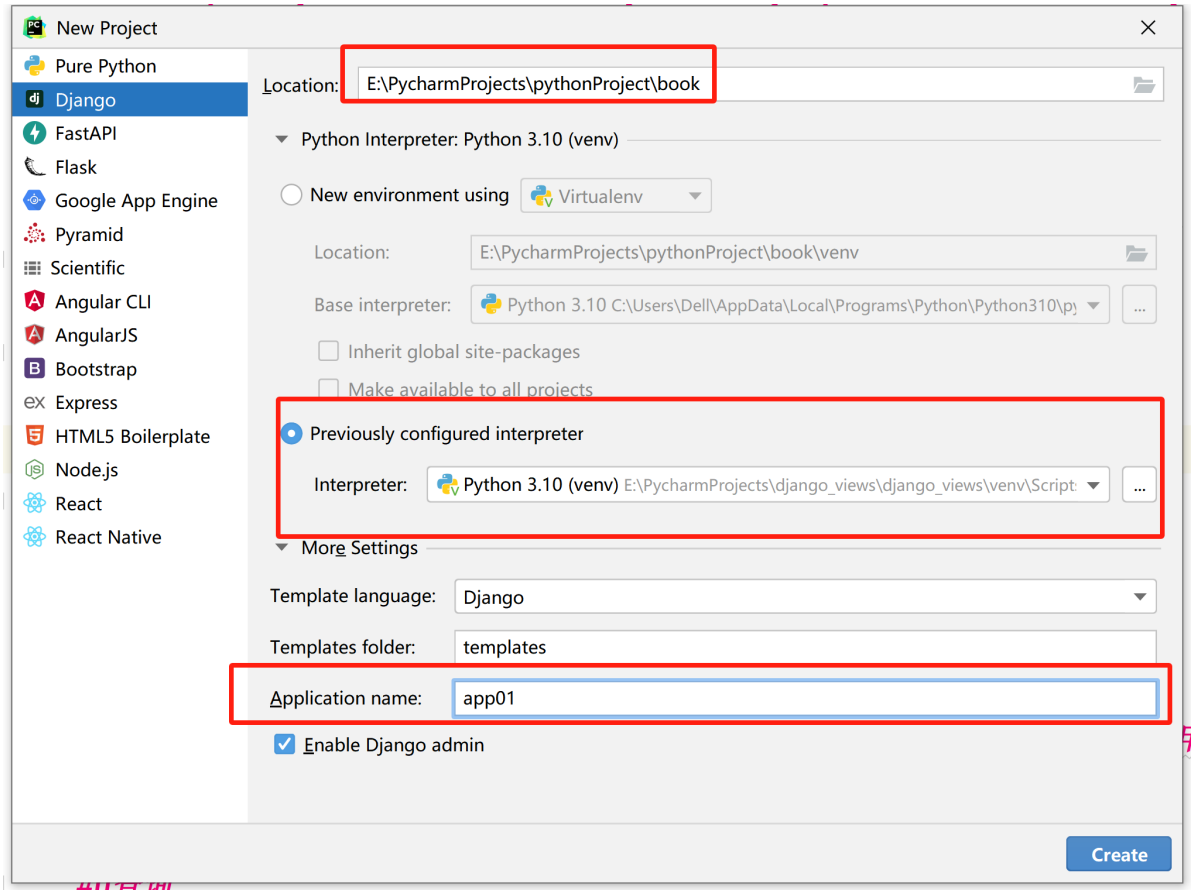
3.0.1 代码演示:

```
#-------------------------------------------------------------
#使用F查询
from django.db.models import F,Q
# 查询卖出数大于库存数的书籍
#select * from book where maichu>kucun;
#filter里的条件可以写多个,用逗号分隔,表示多个条件的and连接
#filter 目前写的一个值 如果有多个 就要用F函数生出一个值
res = models.Book.objects.filter(maichu__gt=F('kucun'))
print(res)

#将所有书籍的价格上调10元
res = models.Book.objects.update(price = F('price')+10) #+ 只能用于数字求和
print(res)

#Q查询
# 多个条件之间的连接(and,or,not)
# 可以使用Q对象将条件包裹之后进行与或非的连接
# &和,都 表示and
# | 表示or
# ~ 表示not

#查询价格高于20元并且卖出数量大于20的书籍 ,号就是and 在这里表示
res = models.Book.objects.filter(price__gt=20,maichu__gt=20)
#如果要使用其他的符号 就只能使用Q函数连接  比如 | ~ &
res1 = models.Book.objects.filter(Q(price__gt=20) & Q(maichu__gt=20))
print(res)
print(res1)
```

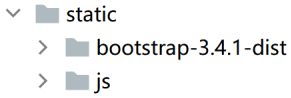
四、图书管理系统

4.1 新建项目

使用软件创建



4.2 添加static



4.3 修改配置 setting.py

```
#暴露static文件夹的路径
STATICFILES_DIRS=[
    BASE_DIR / 'static'
]
```

4.4 创建数据库以及修改数据库配置

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'django_book',
        'USER': 'root',
        'PASSWORD':'wolfcode',
        'PORT':3306,
        'HOST':'127.0.0.1'

    }
}
```

4.5 编写应用中的models.py

```
from django.db import models

# Create your models here.


class Book(models.Model):
    title = models.CharField(max_length=32)
    price = models.DecimalField(max_digits=8,decimal_places=2)
    publish_date = models.DateField(auto_now_add=False)

    #书籍与出版社是一对多
    publish = models.ForeignKey(to='Publish', on_delete=models.CASCADE)
    #书籍与作者是多对多
    authors = models.ManyToManyField(to='Author')

#出版社表
class Publish(models.Model):
    name = models.CharField(max_length=32) #出版社名称
    address = models.CharField(max_length=54) #出版社地址
    email = models.EmailField()  #出版社邮箱

#作者表
class Author(models.Model):
    name = models.CharField(max_length=32) #作者名字
    age = models.IntegerField() #作者年龄

#作者详情表
class AuthorDetail(models.Model):
    phone = models.BigIntegerField() #手机号
    addr = models.CharField(max_length=64) #地址
    #作者和作者详情是一对一
    author = models.OneToOneField(to='Author',on_delete=models.CASCADE)
```

- 生成迁移文件

```
python manage.py makemigrations
```

- 执行迁移

```
python manage.py migrate
```

4.6 初始数据

`app01_publish`

```sql
INSERT INTO `django_book`.`app01_publish`(`id`, `name`, `address`, `email`)
VALUES (1, '东方出版社', '东方', '123@163.com');
INSERT INTO `django_book`.`app01_publish`(`id`, `name`, `address`, `email`)
VALUES (2, '南方出版社', '南方', '456@163.com');
```

`app01_book`

```sql
INSERT INTO `django_book`.`app01_book`(`id`, `title`, `price`, `publish_date`,
`publish_id`) VALUES (1, '三国演义', 36.36, '1999-01-02', 1);
INSERT INTO `django_book`.`app01_book`(`id`, `title`, `price`, `publish_date`,
`publish_id`) VALUES (2, '水浒传', 37.37, '2000-01-01', 1);
INSERT INTO `django_book`.`app01_book`(`id`, `title`, `price`, `publish_date`,
`publish_id`) VALUES (3, '西游记', 40.11, '2004-01-02', 2);
INSERT INTO `django_book`.`app01_book`(`id`, `title`, `price`, `publish_date`,
`publish_id`) VALUES (4, '红楼梦', 66.72, '2008-08-08', 2);
```

`app01_author`

```sql
INSERT INTO `django_book`.`app01_author`(`id`, `name`, `age`) VALUES (1,
'zhangsan', 20);
INSERT INTO `django_book`.`app01_author`(`id`, `name`, `age`) VALUES (2, 'lisi',
26);
INSERT INTO `django_book`.`app01_author`(`id`, `name`, `age`) VALUES (3,
'wangwu', 20);
```

`app01_authordetail`

```sql
INSERT INTO `django_book`.`app01_authordetail`(`id`, `phone`, `addr`,
`author_id`) VALUES (1, 13008833777, '上海 ', 1);
INSERT INTO `django_book`.`app01_authordetail`(`id`, `phone`, `addr`,
`author_id`) VALUES (2, 13677778888, '北京', 2);
INSERT INTO `django_book`.`app01_authordetail`(`id`, `phone`, `addr`,
`author_id`) VALUES (3, 15588889999, '深圳', 3);
```

`app01_book_authors`

```sql
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (1, 1, 1);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (2, 1, 2);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (3, 2, 2);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (4, 2, 3);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (5, 3, 1);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (6, 3, 2);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (7, 4, 1);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (8, 4, 2);
INSERT INTO `django_book`.`app01_book_authors`(`id`, `book_id`, `author_id`)
VALUES (9, 4, 3);
```

4.7 编写首页

book -> urls.py

```
re_path('^$',views.home)
```

app01 -> views.py

```
def home(request):
    return render(request,'home.html')
```

home.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
    {% load static %}

    <!-- 导入外部的css-->
    <link rel="stylesheet" href={% static 'bootstrap-3.4.1-
dist/css/bootstrap.min.css' %}>
    <!-- 导入外部的js-->
    <script src={% static "js/jquery-3.7.0.min.js" %}></script>
    <!-- 导入外部的js-->
    <script src={% static "bootstrap-3.4.1-dist/js/bootstrap.min.js" %}>
</script>

</head>
<body>
<nav class="navbar navbar-default">
  <div class="container-fluid">
```

```
    <!-- Brand and toggle get grouped for better mobile display -->
    <div class="navbar-header">
      <button type="button" class="navbar-toggle collapsed" data-
toggle="collapse" data-target="#bs-example-navbar-collapse-1" aria-
expanded="false">
        <span class="sr-only">Toggle navigation</span>
        <span class="icon-bar"></span>
        <span class="icon-bar"></span>
        <span class="icon-bar"></span>
      </button>
      <a class="navbar-brand" href="#">Brand</a>
    </div>

    <!-- Collect the nav links, forms, and other content for toggling -->
    <div class="collapse navbar-collapse" id="bs-example-navbar-collapse-1">
      <ul class="nav navbar-nav">
        <li class="active"><a href="#">Link <span class="sr-only">(current)
</span></a></li>
        <li><a href="#">Link</a></li>
{#        <li class="dropdown">#}
{#          <a href="#" class="dropdown-toggle" data-toggle="dropdown"
role="button" aria-haspopup="true" aria-expanded="false">Dropdown <span
class="caret"></span></a>#}
{#          <ul class="dropdown-menu">#}
{#            <li><a href="#">Action</a></li>#}
{#            <li><a href="#">Another action</a></li>#}
{#            <li><a href="#">Something else here</a></li>#}
{#            <li role="separator" class="divider"></li>#}
{#            <li><a href="#">Separated link</a></li>#}
{#            <li role="separator" class="divider"></li>#}
{#            <li><a href="#">One more separated link</a></li>#}
{#          </ul>#}
{#        </li>#}
      </ul>
{#      <form class="navbar-form navbar-left">#}
{#        <div class="form-group">#}
{#          <input type="text" class="form-control" placeholder="Search">#}
{#        </div>#}
{#        <button type="submit" class="btn btn-default">Submit</button>#}
{#      </form>#}
      <ul class="nav navbar-nav navbar-right">
        <li><a href="#">Link</a></li>
        <li class="dropdown">
          <a href="#" class="dropdown-toggle" data-toggle="dropdown"
role="button" aria-haspopup="true" aria-expanded="false">Dropdown <span
class="caret"></span></a>
          <ul class="dropdown-menu">
            <li><a href="#">Action</a></li>
            <li><a href="#">Another action</a></li>
            <li><a href="#">Something else here</a></li>
            <li role="separator" class="divider"></li>
            <li><a href="#">Separated link</a></li>
          </ul>
        </li>
      </ul>
    </div><!-- /.navbar-collapse -->
  </div><!-- /.container-fluid -->
</nav>
```

```
<div class="container-fluid">
    <div class="row">
        <!--左边三份 -->
        <div class="col-md-3">
           <div class="list-group">
                          <a href="" class="list-group-item active">首页</a>
                          <a href="#" class="list-group-item">图书列表</a>
                          <a href="#" class="list-group-item">作者列表</a>
                          <a href="#" class="list-group-item">出版社列表</a>
                        </div>
        </div>
        <!--右边三份 -->
        <div class="col-md-9">
            <div class="panel panel-default">
                <div class="panel-heading">图书管理系统</div>
                <div class="panel-body">
                      {% block content %}
                            <div class="jumbotron">
                                 <h1>Hello, world!</h1>
                                  <p>...</p>
                                  <p><a class="btn btn-primary btn-lg" href="#"
role="button">Learn more</a></p>
                             </div>
                         {% endblock %}
                </div>
            </div>

        </div>
    </div>

</div>
</body>
</html>
```

4.8 编写图书列表

book -> urls.py

```
#图书列表
path('book/list',views.book_list,name='book_list'),
```

app01 -> views.py

```
def book_list(request):
#查询所有的图书列表
book_queryset = models.Book.objects.all()

return render(request,'book_list.html',locals())
```

book_list.html

```
{% extends 'home.html' %}
```

```
{% block content %}
    <h1 class="text-center">图书列表</h1>
    <table class="table table-hover table-striped">
        <thead>
            <tr>
                <th>ID</th>
                <th>书名</th>
                <th>作者</th>
                <th>价格</th>
                <th>出版时间</th>
                <th>出版社</th>
                <th>操作</th>
            </tr>
        </thead>
        <tbody>
            {% for book_query in book_queryset %}
                <tr>
                    <td>{{ book_query.pk }}</td>
                    <td>{{ book_query.title }}</td>
                    <td>{{ book_query.authors.all }}</td>
                    <td>{{ book_query.price }}</td>
                    <td>{{ book_query.publish_date }}</td>
                    <td>{{ book_query.publish.name }}</td>
                    <td>
                        <a href="" class="btn btn-primary btn-xs">编辑</a>
                        <a href="" class="btn btn-danger  btn-xs">删除</a>
                    </td>
                </tr>
            {% endfor %}

        </tbody>

    </table>
{% endblock %}
```

单独测试页面后

发现作者渲染出来的数据有问题 渲染出来得到的是对象 没渲染具体的名字

发现日期也没格式化

修改:

```
{% for book_query in book_queryset %}
                <tr>
                    <td>{{ book_query.pk }}</td>
                    <td>{{ book_query.title }}</td>
```

```
                    <td>
                        {% for author_obj in book_query.authors.all %}
                              {% if not forloop.first %} | {% endif %}{{
author_obj.name }}
                        {% endfor %}
                    </td>
                    <td>{{ book_query.price }}</td>
                    <td>{{ book_query.publish_date|date:'Y-m-d' }}</td>
                    <td>{{ book_query.publish.name }}</td>
                    <td>
                        <a href="" class="btn btn-primary btn-xs">编辑</a>
                        <a href="" class="btn btn-danger  btn-xs">删除</a>
                    </td>
                </tr>
{% endfor %}
```

4.9 添加图书

boot_list.html

```
<a href="" class="btn btn-success pull-left">添加图书</a>
```

book -> urls.py

```
#添加图书
path('book/add',views.book_add,name='book_add'),
```

app01 -> views.py

```
#图书添加
def book_add(request):
    return render(request, 'book_add.html')
```

book_list.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">添加图书</h2>
    <form action="" method="post">
        <p>书名：<input type="text" name="title" class="form-control"></p>
        <p>价格：<input type="text" name="price" class="form-control"></p>
        <p>出版时间：<input type="date" name="publish_date" class="form-control">
</p>
        <p>出版社：
                <select name="publish">
                    {% for publish_obj in publish_queryset %}
                            <option value="{{ publish_obj.pk }}">{{
publish_obj.name }}</option>
                    {% endfor %}
                </select>
        </p>
        <p>作者：
               {% for author_obj in author_queryset %}
```

```
                    <input type="checkbox" name="author" value="{{ author_obj.pk
}}">{{ author_obj.name }}
                {% endfor %}

        </p>
        <input type="submit" class="btn btn-success" value="提交">

    </form>
{% endblock %}
```

book_list.html

```
<a href="{% url 'book_add' %}" class="btn btn-success pull-left">添加图书</a>
```

点击提交

```
#图书添加
def book_add(request):
    #查询所有的作者
    author_queryset = models.Author.objects.all()
    #查询所有的出版社
    publish_queryset = models.Publish.objects.all()

    if request.method == 'POST':
        #标题
        title = request.POST.get('title')
        #价格
        price = request.POST.get('price')
        #出版社日期
        publish_date = request.POST.get('publish_date')
        #出版社ID
        publish_id = request.POST.get('publish')
        #作者ID 因为有多个值 要用列表
        author_ids = request.POST.getlist('author')
        #保存数据到数据库
        book_obj = models.Book.objects.create(title=title,
                                              price=price,
                                              publish_date=publish_date,
                                              publish_id=publish_id)
        #create()会将创建的对象的返回
        #保存第三张表的数据
        #add() 可以传入多个值 *列表表示将列表打散成位置参数
        book_obj.authors.add(*author_ids)
        return redirect('book_list')

    return render(request, 'book_add.html',locals())
```

完成测试

4.10 编辑图书

book_list.html

```
<a href="{% url 'book_edit' edit_id=book_query.pk %}" class="btn btn-primary
btn-xs">编辑</a>
```

book - > urls.py

```
#编辑图书
re_path('book/edit/(?P<edit_id>\d+)', views.book_edit, name='book_edit'),
```

app01 - > views.py

```
#编辑
def book_edit(request,edit_id):
    #根据id获取图书对象
    book_obj = models.Book.objects.filter(pk=edit_id).first()
    # 查询所有的作者
    author_queryset = models.Author.objects.all()
    # 查询所有的出版社
    publish_queryset = models.Publish.objects.all()

    return render(request,'book_edit.html',locals())
```

templates -> book_edit.html

```
{% extends 'home.html' %}

{% block content %}
    <h2 class="text-center">编辑图书</h2>
    <form action="" method="post">
        <p>书名：<input type="text" value="{{ book_obj.title }}" name="title"
class="form-control"></p>
        <p>价格：<input type="text" value="{{ book_obj.price }}" name="price"
class="form-control"></p>
        <p>出版时间：<input type="date" value="{{ book_obj.publish_date|date:'Y-m-
d' }}" name="publish_date" class="form-control"></p>
        <p>出版社：
                <select name="publish">
                    {% for publish_obj in publish_queryset %}
```

```
                                <option {% if publish_obj == book_obj.publish
%}checked{% endif %} value="{{ publish_obj.pk }}">{{ publish_obj.name }}
</option>
                        {% endfor %}
                    </select>
            </p>
            <p>作者:
                    {% for author_obj in author_queryset %}
                        <input {% if author_obj in book_obj.authors.all %}checked{%
endif %}  type="checkbox" name="author" value="{{ author_obj.pk }}">{{
author_obj.name }}
                    {% endfor %}

            </p>
            <input type="submit" class="btn btn-success" value="提交">

        </form>
{% endblock %}
```

修改 -> 点击提交

app01 - > views.py

```
#编辑
def book_edit(request,edit_id):
    #根据id获取图书对象
    book_obj = models.Book.objects.filter(pk=edit_id).first()
    # 查询所有的作者
    author_queryset = models.Author.objects.all()
    # 查询所有的出版社
    publish_queryset = models.Publish.objects.all()

    if request.method == 'POST':
        #标题
        title = request.POST.get('title')
        #价格
        price = request.POST.get('price')
        #出版社日期
        publish_date = request.POST.get('publish_date')
        #出版社ID
        publish_id = request.POST.get('publish')
        #作者ID 因为有多个值 要用列表
        author_ids = request.POST.getlist('author')

        #修改图书表
        book_obj.title = title
        book_obj.price = price
        book_obj.publish_date = publish_date
        book_obj.publish_id=publish_id
        #修改第三张关系表
        book_obj.authors.set(author_ids) #set() 必须传入一个可迭代对象

        book_obj.save()
        return redirect('book_list')

    return render(request,'book_edit.html',locals())
```

完成测试

4.11 删除图书

book_list.html

```
<a href="{% url 'book_delete' delete_id=book_query.pk %}" class="btn btn-danger btn-xs">删除</a>
```

book - > urls.py

```
#删除图书
re_path('book/delete/(?P<delete_id>\d+)', views.book_delete, name='book_delete'),
```

app01 - > views.py

```
#删除图书
def book_delete(request,delete_id):
    #根据id直接删除
    book_obj = models.Book.objects.filter(pk=delete_id).delete()
    return redirect('book_list')
```

五、作业

5.1 完成本天的案例